From Raw NASA Feeds to a Prediction Pipeline

Feature engineering, model-readiness, and where the data plugs into AgriFusionBD

11 November 2026. Every number in this report is generated by the scripts named in section 9 and stored in Data/model_ready/ and Data/_report/.

1. Summary

The six NASA datasets were turned into two analysis tables: a daily panel of 12,313 rows (7 regions x 1,759 days) and a monthly panel of 406 rows, carrying 90 documented features. Every feature is computed only from information available at the end of its own day, every target is an explicit forward shift, and nothing is zero-filled.

The tables predict what should be predictable and refuse what should not. Next-day soil moisture reaches R2 = 0.67 and next-day reference ET0 R2 = 0.58, both well above a persistence baseline. Next-day rainfall stays near zero: a single point, one day ahead, from these predictors, is not a solved problem and the pipeline does not pretend otherwise.

One serious defect had to be fixed before any of this was trustworthy: the existing POWER downloads carry precipitation multiplied by exactly 24. Corrected copies are in Data/power_corrected/; the originals were left untouched.

2. A defect that had to be fixed first

While building rainfall features, POWER precipitation summed to 197,188 mm over four years at Rajshahi — about 112 mm every day, including the dry season. Checking the same point and days against the live POWER API gave a daily value 24 times smaller, while the other seven parameters matched exactly:

Parameter	Live POWER API	Local CSV	Ratio
T2M	27.05 C	27.05 C	1.000
T2M_MAX	29.01 C	29.01 C	1.000
T2M_MIN	25.53 C	25.53 C	1.000
RH2M	93.79 %	93.79 %	1.000
WS2M	4.04 m/s	4.04 m/s	1.000
ALLSKY_SFC_SW_DWN	5.98 MJ/m2	5.98 MJ/m2	1.000
PS	100.34 kPa	100.34 kPa	1.000
PRECTOTCORR	77.41 mm	1,857.84 mm	24.000

Table 1. Northwest Rajshahi, 4 October 2023. 40 further samples across all seven regions and ten dates in every season gave a ratio of exactly 24.00, so this is a constant scaling error, not noise.

The fix is mechanical: Data/power_corrected/ holds copies with PRECTOTCORR divided by 24 (the raw column is preserved beside it as PRECTOTCORR_raw), and the feature builder prefers those copies. Corrected annual totals now run from 1,710 mm at Rajshahi to 5,200 mm at Sylhet, in line with the satellite rainfall record. Anything previously computed from the original POWER files — irrigation demand in particular — should be recomputed.

3. How the features were built

- **No future information.** Each row's features use data up to and including that day. Targets are shifted forward and are never filled.

- **Missing stays missing.** No zero-filling. Where a sparse satellite variable is carried forward it gets an age column and an observed-today flag, so a model can separate a fresh retrieval from a two-week-old one.
- **Bounded carry-forward.** HLS 45 days, ECOSTRESS 10, GRACE 60. Past the cap the value becomes null rather than being silently reused.
- **Time-based splits.** Train 2022-2024, validation 2025, test 2026. Daily environmental series are strongly autocorrelated, so a random split would leak the future.

Feature family	Examples	Built from
Weather	t2m_c, tmax_c, rh_pct, solar_mj_m2, pressure_kpa	POWER (corrected)
Derived meteorology	et0_best_mm, vpd_kpa, gdd_base10, heat_day_35, diurnal_range_c	FAO-56 Penman-Monteith; Hargreaves fallback
Rainfall windows	power_rain_sum_3d ... _90d, gpm_rain_sum_30d, dry_spell_days	Rolling sums ending on the row date
Soil moisture	sm_mean, sm_7d_mean, sm_anomaly_30d, sm_obs_age_days	SMAP AM/PM retrievals
Vegetation	ndvi_last, ndmi_last, hls_age_days	HLS, carried at most 45 days
Surface temperature	lst_c_last, ecostress_age_days	ECOSTRESS clear-sky overpasses
Water storage	grace_lwe_cm_last, grace_age_days	GRACE mascon, 60-day carry
Terrain	elevation_mean_m, slope_mean_deg	SRTM, static
Calendar	doy_sin, doy_cos, month, is_monsoon, season	Derived; season follows the plan's boro/aman/rabi windows

Table 2. The feature families in model_ready/. The full per-column dictionary is in feature_dictionary.csv.

Two derived variables deserve a note. ET0 is computed twice: the full FAO-56 Penman-Monteith equation where POWER supplies solar radiation, and a Hargreaves fallback where it does not (86 days per region). A flag records which was used, because the two are not interchangeable. VPD is derived from temperature and humidity rather than stored by any product.

4. What is in model_ready/

File	Rows	Contents
daily_panel.csv	12,313	7 regions x 1,759 days, 64 columns
monthly_panel.csv	406	7 regions x 58 months, 37 columns
feature_dictionary.csv	97	Every column: source, unit, meaning, leakage note
data_quality_report.csv	92	Completeness per column
split_manifest.json	-	Train/valid/test boundaries
baseline_results.json	-	Scores of the reference models
README.md	-	Usage rules and caveats

Table 3. Outputs of build_model_ready.py.

Targets available, with the number of rows that actually carry a value:

Target	Table	Rows with a value	Why it is useful
target_sm_next_day	daily	8,025	Irrigation cue when no rover is in the field
target_et0_next_day_mm	daily	12,285	Water demand, feeding the engine's water balance
target_rain_next_day_mm	daily	12,306	Spray/fertiliser timing, dry-spell onset

target_heat_next_day	daily	12,285	Heat-stress warning
ndvi_next_month	monthly	214	Canopy trend for rotation comparison

Table 4. Prediction targets.

5. Can these features predict anything?

train_baseline.py fits persistence and climatology baselines, then ridge and gradient boosting, on the time-based split. Reporting baselines beside the models matters: a model that cannot beat 'tomorrow looks like today' has added nothing.

Target	Model	MAE	RMSE	R2
Rainfall, next day (mm)	persistence	5.700	12.560	-0.15
	climatology	7.358	12.225	-0.09
	ridge	6.861	11.311	0.07
	gradient boosting	7.634	12.811	-0.20
Soil moisture, next day (m3/m3)	persistence	0.074	0.098	0.21
	climatology	0.091	0.112	0.07
	ridge	0.054	0.070	0.64
	gradient boosting	0.051	0.066	0.67
ET0, next day (mm)	persistence	0.468	0.692	0.46
	climatology	0.745	1.013	-0.16
	ridge	0.446	0.610	0.58
	gradient boosting	0.500	0.672	0.49
NDVI, next month	persistence	0.087	0.129	0.27
	climatology	0.104	0.128	0.12
	ridge	0.067	0.089	0.57
	gradient boosting	0.089	0.115	0.28

Table 5. Test-period scores (2026 hold-out). Soil moisture and ET0 are clearly predictable; rainfall is not, and the NDVI test set is only 26 monthly rows, so its score is indicative at best.

The honest reading: next-day soil moisture carries real skill (R2 0.67 against 0.21 for persistence) because the surface retains water from one day to the next. ET0 likewise, because it is a smooth function of weather. Rainfall one day ahead is close to noise at a point: the features describe the atmosphere's state, not its triggering. Any product built on this should present rainfall as climatological risk and current wetness, not as a deterministic forecast.

6. What drives the signal

Target	Rows	Strongest features
NDVI, next month	139	precip_power_m, t2m_min_m, month_cos, t2m_range_m
Soil moisture, next day	4535	smap_am, smap_am_ffill3d, ndmi
Rain occurrence, next day	12306	rain_days_7d
Rain amount, next day	12306	rain_days_7d, precip_power, gpm_occ

Table 6. From the independent screening pass (Data/_report/feature_screening.json).

The pattern is consistent across targets and methods: recent wetness dominates. For soil moisture the previous days' retrievals carry almost all of the signal; for rainfall, the count of rainy days in the last week is the strongest single predictor; for monthly NDVI, temperature extremes and accumulated rainfall matter more than any instantaneous value. This is why the feature set leans on rolling windows rather than single-day snapshots.

7. Where this plugs into AgriFusionBD

The backend already has the right shape: apps/environment owns DatasetProduct, DataAsset and FieldIndicator, and apps/analyses/engine is a pure calculation layer that consumes an EngineInput. No new Django app is needed for any of these datasets.

Dataset	Engine hook that exists	What to extend
Rainfall	PowerDay.prectot -> _effective_rain (water_balance.py)	accept a GPM series as an alternative source
Soil moisture	soil bucket starts full (water_balance.py:67)	initialise the bucket from SMAP; gate dry_spell_days
Land-surface temperature	heat.py uses air tmax only	add lst to PowerDay and use it in season_heat
Terrain	elevation_m is accepted but never passed	pass SRTM elevation at water_balance.py:75; slope into bucket capacity
Water storage	none	new indicator kind; initialise bucket from GRACE
Vegetation	cover.py takes NDVI + coverage	add NDMI alongside NDVI

Table 7. Dataset-to-engine mapping. Full detail with line references is in Data/_report/agrifusion_map.md.

The shortest path to visible progress, in order:

- **1. Extend the indicator kinds.** Add soil_moisture, land_surface_temperature, ndmi, water_storage, elevation and slope to FieldIndicator.KIND_CHOICES, then migrate.
- **2. Write one service per source.** Copy the shape of environment/services/power.py: product, content-hashed asset, indicators, nulls for gaps.
- **3. Thread one dataset through the engine first.** Soil moisture into water_balance is the highest-value change: it turns a bucket that starts full into one grounded in observation.
- **4. Version the rules.** If a metric definition changes, bump RULE_VERSION so old analyses stay explainable.

8. Problems in the current backend worth fixing

Found by reading the repository, not by running it. They matter because new data sources will copy these patterns.

- **ET0 is labelled FAO-56 but is not.** engine/et0.py builds net longwave from |T - 273|^4 and ignores humidity; the correct series in model_ready/ can replace it. Irrigation demand inherits the error.
- **Power indicator persistence looks broken.** _store_indicators omits period_end while the column is non-null with no default; a fresh live fetch would fail on the first insert.
- **NDVI never reaches the API.** The HLS service writes a DataAsset but no FieldIndicator rows, so vegetation data exists but is not queryable through the indicators endpoint.
- **The asset cache key excludes the field.** DataAsset.cache_key is globally unique and field-free while DataAsset.field records only the first caller, so per-field provenance can point at another field's asset.
- **Engine tests assert on wording.** A water-balance qualification must contain 'uncertainty' or 'not zero', so rewording breaks tests.

- **Metric keys are coupled in four places.** A key missing from rules.BOUNDS silently normalises to zero instead of raising.

9. Reproducing this

All scripts live in Data/ and run with Data/.venv:

Script	Does
fix_power_precip.py	writes corrected POWER copies (PRECTOTCORR / 24)
build_model_ready.py	builds both panels, the dictionary and the quality report
train_baseline.py	fits baselines and models, writes baseline_results.json
report_stats.py / report_figures.py	statistics and figures for the data report
build_report_docx.py	builds NASA_Data_Report.docx

Table 8. Pipeline scripts. All are idempotent.

Caveat that applies to every number here: the record is under five years, the seven points are single coordinates, and nothing in these tables is a measurement at a farm. They are regional context, modelled and re-sampled, and should be labelled as such wherever they are shown.